



nomic-ai/modernbert-embed-base

like 233

Follow

Nomic AI

1.22k



Sentence Similarity



sentence-transformers



ONNX



Safetensors



Transformers.js



English

modernbert

feature-extraction

mteb



Eval Results (legacy)



text-embeddings-inference



arxiv:2402.01613



License: apache-2.0



Deploy

→ Copy to bucket

NEW

Use this model



Model card



Files



xet

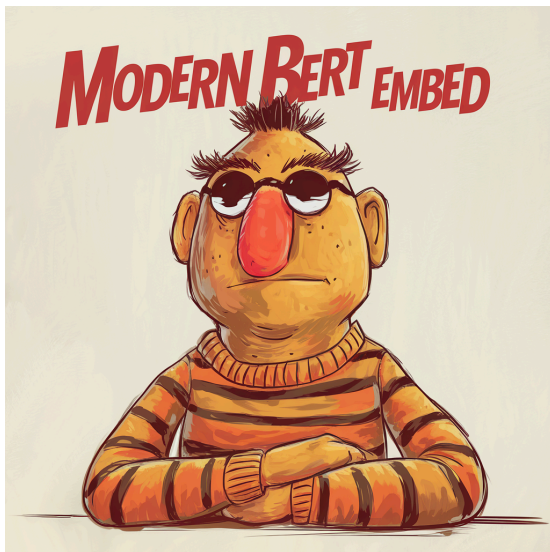


Community

14



ModernBERT Embed



ModernBERT Embed is an embedding model trained from [ModernBERT-base](#), bringing the new advances of ModernBERT to embeddings!

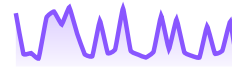
Trained on the [Nomic Embed](#) weakly-supervised and supervised datasets, modernbert-embed also supports Matryoshka Representation Learning dimensions of 256, reducing memory by 3x with minimal performance loss.

Performance

Model	Dimensions	Average (56)	Classification (12)
nomic-embed-text-v1	768	62.4	74.1

Downloads last month

232,798



Safetensors

Model size 0.1B params

Tensor type F32

Files info

Run 15,000+ Models Instantly

Inference Providers

Sentence Similarity

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

Source Sentence

Your sentence here...

nomic-ai/modernbert-embed-base is supported by the following Inference Providers:

Sentences to compare

Your sentence here...

HF HF Inference API

View API Code

Dismiss

Your sentence here...

Add Sentence

Generate

</> View Code

Maximize

Model tree for nomic-ai/modernbe...

Base model [answerdotai/ModernBERT-](#)Finetuned (1460) [this model](#)

Finetunes 115 models



Quantizations 4 models

Spaces using nomic-ai/modern... 18



mteb/leaderboard



nkhokhla/dualuse-academic-assessor



as-cle-bert/PapersChat



eswardivi/ModernBert-Similarity

Model	Dimensions	Average (56)	Classification (12)
nomic-embed-text-v1.5	768	62.28	73.55
modernbert-embed-base	768	62.62	74.31
nomic-embed-text-v1.5	256	61.04	72.1
modernbert-embed-base	256	61.17	72.40

Usage

You can use these models directly with the latest transformers release and requires installing transformers>=4.48.0:

```
pip install transformers>=4.48.0
```

Reminder, this model is trained similarly to Nomic Embed and **REQUIRES** prefixes to be added to the input. For more information, see the instructions in [Nomic Embed](#).

Most use cases, adding search_query: to the query and search_document: to the documents will be sufficient.

Sentence Transformers

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("nomic-ai/modernbert-embed-base")

query_embeddings = model.encode([
    "search_query: What is TSNE?",
    "search_query: Who is Laurens van der Maaten?"
])
doc_embeddings = model.encode([
    "search_document: TSNE is a dimensionality reduction technique."
])

print(query_embeddings.shape, doc_embeddings.shape)
# (2, 768) (1, 768)

similarities = model.similarity(query_embeddings, doc_embeddings)
```

maxpar1/leaderboard

DrishhtiSharma/docqa-with-deepsee...

CadenShokat/modernbert-finetune...

sudormrfbin/binary-shield

+ 10 Spaces

Paper for nomic-ai/modernbert-e...

Nomic Embed: Training a Reproduc...

Paper • 2402.01613 • Publi... • 18

Evaluation results

accuracy ...	test set	self-reported	78.134
ap on MTE...	test set	self-reported	42.190
f1 on MTE...	test set	self-reported	72.344
accuracy ...	test set	self-reported	93.094
ap on MTE...	test set	self-reported	90.037
f1 on MTE...	test set	self-reported	93.087
accuracy ...	test set	self-reported	48.428
f1 on MTE...	test set	self-reported	47.743

Expand 1128 evaluations

```
print(similarities)
# tensor([[0.7214],
#         [0.3260]])
```

- Click to see Sentence Transformers usage with Matryoshka Truncation

Transformers

```
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

def mean_pooling(model_output, attention_mask):
    token_embeddings = model_output[0]
    input_mask_expanded = (
        attention_mask.unsqueeze(-1).expand(
            token_embeddings.size()
        )
    )
    return torch.sum(token_embeddings * input_mask_expanded,
        input_mask_expanded.sum(1), min=1e-9
    )

queries = ["search_query: What is TSNE?", "search_query: What is PCA?"]
documents = ["search_document: TSNE is a dimensionality reduction technique",
            "search_document: PCA is a dimensionality reduction technique"]

tokenizer = AutoTokenizer.from_pretrained("nomic-ai/nomic-embedder-v1.5")
model = AutoModel.from_pretrained("nomic-ai/nomic-embedder-v1.5")

encoded_queries = tokenizer(queries, padding=True, return_tensors="pt")
encoded_documents = tokenizer(documents, padding=True, return_tensors="pt")

with torch.no_grad():
    queries_outputs = model(**encoded_queries)
    documents_outputs = model(**encoded_documents)

query_embeddings = mean_pooling(queries_outputs, encoded_queries["attention_mask"])
query_embeddings = F.normalize(query_embeddings, p=2, dim=-1)
doc_embeddings = mean_pooling(documents_outputs, encoded_documents["attention_mask"])
doc_embeddings = F.normalize(doc_embeddings, p=2, dim=-1)
print(query_embeddings.shape, doc_embeddings.shape)
# torch.Size([2, 768]) torch.Size([1, 768])

similarities = query_embeddings @ doc_embeddings
print(similarities)
# tensor([[0.7214],
#         [0.3260]])
```

- Click to see Transformers usage with Matryoshka Truncation

Transformers.js

If you haven't already, you can install the

Transformers.js JavaScript library from NPM using:

```
npm i @huggingface/transformers
```

Then, you can compute embeddings as follows:

```
import { pipeline, matmul } from '@huggingface/transformers'

// Create a feature extraction pipeline
const extractor = await pipeline(
  "feature-extraction",
  "nomic-ai/modernbert-embed-base",
  { dtype: "fp32" }, // Supported options: "float16", "float32"
);

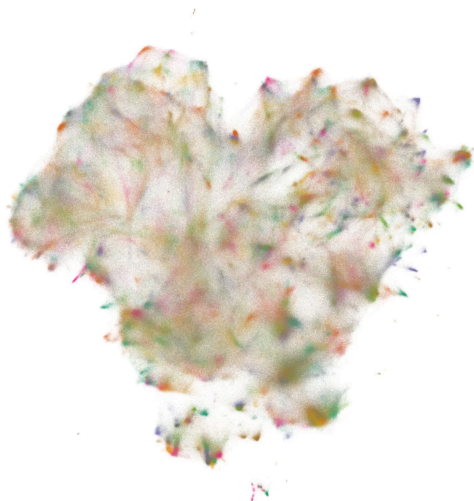
// Embed queries and documents
const query_embeddings = await extractor([
  "search_query: What is TSNE?",
  "search_query: Who is Laurens van der Maaten?",
], { pooling: "mean", normalize: true },
);

const doc_embeddings = await extractor([
  "search_document: TSNE is a dimensionality reduction technique.",
], { pooling: "mean", normalize: true },
);

// Compute similarity scores
const similarities = await matmul(query_embeddings, doc_embeddings);
console.log(similarities.tolist()); // [[0.7, 0.6]]
```

Training

Click the Nomic Atlas map below to visualize a 5M sample of our contrastive pretraining data!



We train our embedder using a multi-stage training pipeline. Starting from a long-context [BERT model](#), the first unsupervised contrastive stage trains on a dataset generated from weakly related text pairs, such as question-answer pairs from forums like StackExchange and Quora, title-body pairs from Amazon reviews, and summarizations from news articles.

In the second finetuning stage, higher quality labeled datasets such as search queries and answers from web searches are leveraged. Data curation and hard-example mining is crucial in this stage.

For more details, see the Nomic Embed [Technical Report](#) and corresponding [blog post](#).

Training data to train the models is released in its entirety. For more details, see the [contrastors repository](#).

Join the Nomic Community

- Nomic: <https://nomic.ai>
- Discord: <https://discord.gg/myY5YDR8z8>
- Twitter: https://twitter.com/nomic_ai

Citation

If you find the model, dataset, or training code useful, please cite our work

```
@misc{nussbaum2024nomic,  
  title={Nomic Embed: Training a Reproducible  
  author={Zach Nussbaum and John X. Morris},  
  year={2024},  
  eprint={2402.01613},  
  archivePrefix={arXiv},  
  primaryClass={cs.CL}}
```

 System theme

[TOS](#)

[Privacy](#)

[About](#)

[Careers](#)



[Models](#)

[Datasets](#)

[Spaces](#)

[Pricing](#)

[Docs](#)